



کدهای مربوط به هر آزمایش در فایل `az1.m` به صورت بخش بندی شده با کامنت ارسال شده است.

در این گزارشکار اسکرین شات از اجرای کدها و توضیحات مربوطه قرار دارد.

-۱

فعالیت ۱: برنامه ای بنویسید که یک عدد مختلط از کاربر دریافت کرده، اندازه دامنه و زاویه عبارت زیر را محاسبه کرده و نتیجه را نمایش دهد.

$$y = x^2 + 3x + 10$$

```
x = input("Enter a number:");
y = x^2 + 3*x + 10;

ang = angle(y);
magnitude = abs(y);

disp(['Result = ' num2str(y)]);
fprintf("Angle (rad) = %f\n", ang);
fprintf("Angle (degree) = %f\n", rad2deg(ang));
fprintf("Magnitude = %f\n", magnitude);
```

```
Enter a number:2+2i
Result = 16+14i
Angle (rad) = 0.718830
Angle (degree) = 41.185925
Magnitude = 21.260292
```

-۲

فعالیت ۲: با استفاده از اپراتور : اعداد زوج دو رقمی را تولید کنید. با دستور `linspace` اعداد زوج یک رقمی را تولید کنید. سپس دو آرایه ایجاد شده را با هم `concat` کنید.

```
zgj_do_ragham = 10:2:99;
zgj_yek_ragham = linspace(0, 8, 5);

zgj = [zgj_yek_ragham, zgj_do_ragham];
disp(['zgj yek ragham = ' num2str(zgj_yek_ragham)])
disp(['zgj do ragham = ' num2str(zgj_do_ragham)])
disp(['zgj concatenation= ' num2str(zgj)])
```

-۳

$$y = |\sin(x)| + 5x^3 + 20$$

x =	-18.8496	-18.7496	-18.6496	-18.5496	-18.4496	-18.3496	-18.2496	-18.1496	-18.0496
y =	-33466.7788	-32936.5428	-32411.9326	-31892.9193	-31379.4738	-30871.567	-30369.1699	-298	

-۴

$$\begin{bmatrix} 1 & \dots & 10 \\ \vdots & \ddots & \vdots \\ 1 & \dots & 10 \end{bmatrix}_{10 \times 10}$$
[illegible]

فعالیت ۵: برنامه‌ای بنویسید که ماتریس دو ستونی از کاربر دریافت کند، به نحوی که ستون اول نمرات ۵ درس یک ترم دانشجو باشد و ستون دوم تعداد واحد هر درس، سپس عملیات زیر را بر روی آن انجام دهد:

(۱) محاسبه تعداد واحدها

(۲) محاسبه معدل

(۳) نمایش نتیجه با پیام مناسب

```
% mat = [ 19, 3
%         20, 3
%         15, 3
%         17, 3
%         20, 1 ];

mat = input("enter a mat: ");
vahedha = sum(mat(:, 2));
moadel = sum(mat(:,1) .* mat(:,2)) / vahedha;

disp(['Vahedha: ' num2str(vahedha)]);
disp(['Moadel: ' num2str(moadel)]);

enter a mat: [19, 3; 20, 3; 15, 3; 17, 3 ; 20, 1;]
Vahedha: 13
Moadel: 17.9231
```

فعالیت ۶: آشنایی با دستورات for, if, while

```
x = [];
for n = 1:10
    x(n) = sin(n * pi / 10)
end

Ages = input('Ages = ');
k = length(Ages);
for i = 1:k
    if Ages(i) < 17
        disp('Teen')
    elseif Ages(i) > 17 && Ages(i) < 40
        disp('Young')
    elseif Ages(i) > 40 && Ages(i) < 60
        disp('Middle - aged')
    end
end

x = [];
```

```

for n=1:10
    x(n) = sin(n * pi / 10);
end

disp(['x = ' num2str(x)])

ages = input('Ages = ');
k = length(ages);

for i=1:k
    if ages(i) < 17
        disp('Teen');
    elseif ages(i) > 17 && ages(i) < 40
        disp('Young');
    elseif ages(i) > 40 && ages(i) < 60
        disp('Middle-aged');
    end
end

x = 0.30902    0.58779    0.80902    0.95106    1    0.95106    0.80902    0.58779    0.30902    1.2246e-16
Ages = [12 14 15 16 20 12 21 31 23 43 53 58]
Teen
Teen
Teen
Teen
Young
Teen
Young
Young
Young
Middle-aged
Middle-aged
Middle-aged
^^

```

-۷

فعالیت ۷: تابعی بنویسید که نمرات ۱۰ دانشجوی یکسان را به همراه شماره دانشجویی از ۵ استاد بعنوان آرگومان ورودی دریافت کرده و میانگین نمرات هر درس، و معدل هر دانشجو را بعنوان خروجی تولید کرده و نمایش دهد. راهنمایی: ایجاد توابع در متلب:

```

function [out1,out2,out3, ...] = Name(input1,input2,inut3, ...)
    %comments
    Rules
end

function [lesson_avg, student_gpa] = grade_analyzer(scores, student_ids)
    % scores: matrice 10x5 ()
    % student_ids: شماره دانشجویی

    % میانگین نمرات هر درس (میانگین ستون‌ها)
    lesson_avg = mean(scores, 1);

    % معدل هر دانشجو (میانگین سطرها)
    student_gpa = mean(scores, 2);

    % نمایش نتایج
    disp('lesson_avg:');
    disp(lesson_avg);
    disp('Each Student:');

```

```

    for i = 1:length(student_ids)
        disp(['Student ID ', num2str(student_ids(i)), ' GPA: ',
num2str(student_gpa(i))]);
    end
end
scores = randi([10, 20], 10, 5)
ids = 101:110
[avg, gpa] = grade_analyzer(scores, ids);

scores =

    20    10    11    10    12
    16    19    17    18    12
    15    20    15    15    19
    12    18    18    15    10
    15    11    17    19    15
    16    12    19    16    11
    17    13    19    16    20
    14    17    13    19    17
    14    11    17    18    15
    20    17    12    16    15

ids =

    101    102    103    104    105    106    107    108    109    110

lesson_avg:
    15.9000    14.8000    15.8000    16.2000    14.6000

Each Student:
Student ID 101 GPA: 12.6
Student ID 102 GPA: 16.4
Student ID 103 GPA: 16.8
Student ID 104 GPA: 14.6
Student ID 105 GPA: 15.4
Student ID 106 GPA: 14.8
Student ID 107 GPA: 17
Student ID 108 GPA: 16
Student ID 109 GPA: 15
Student ID 110 GPA: 16

```

-۸

فعالیت ۸: یک ماتریس ۳×۳ با درایه‌های تصادفی ایجاد کرده، سپس با استفاده از توابع `det`, `inv`, `pinv`, `trace` و سایر توابع کار با ماتریس نتیجه را مشاهده کنید.

```

mat = randn(3);

trace_mat = trace(mat)
pseudoinverse_mat = pinv(mat)
inverse_mat = inv(mat)
det_mat = det(mat)

```

```

trace_mat =

    -1.5486

pseudoinverse_mat =

    0.2655    0.5974   -0.5988
   -0.7407   -0.4596    0.7181
   -0.1405    1.0186    0.4293

inverse_mat =

    0.2655    0.5974   -0.5988
   -0.7407   -0.4596    0.7181
   -0.1405    1.0186    0.4293

det_mat =

    2.6769

```

بخش دوم: تابع ضربه و تابع پله واحد، سینوسی و تصادفی

-۹

فعالیت ۹: بردارهای زیر را ایجاد کرده، انرژی آنها را محاسبه کرده و با استفاده از ابزار ترسیم در متلب، نتیجه را ترسیم کنید.

توجه: برای ترسیم توابع زمان گسسته عموماً از دستور `stem` استفاده میشود.

توجه: در صورتیکه بخواهیم در سیگنال با طول N ، تاخیر به اندازه M سمپل ایجاد کنیم ($M < N$)، میتوانیم به مقدار تاخیر، صفر به ابتدای سیگنال اضافه کنیم.

```

N = 20;
M = 5;
n = 0:N-1;
A = 2;
c = -0.1;

delta = [1, zeros(1, N-1)];
u = ones(1, N);
delta_M = [zeros(1, M), 1, zeros(1, N - M - 1)];
exponential = A * exp(n * c);

E_delta = sum(delta .* delta);
E_u = sum(u .* u);

```

```

E_delta_M = sum(delta_M .* delta_M);
E_exp = sum(exponential .* exponential);

disp(['انرژی ضربه: ' num2str(E_delta)]);
disp(['انرژی پله: ' num2str(E_u)]);
disp(['انرژی ضربه تاخیر یافته: ' num2str(E_delta_M)]);
disp(['انرژی نمایی: ' num2str(E_exp)]);

figure; grid on;
subplot(2,2,1);
stem(n, delta, 'filled', 'LineWidth', 2);
title('تابع ضربه');

subplot(2,2,2);
stem(n, u, 'filled', 'LineWidth', 2);
title('تابع پله واحد');

subplot(2,2,3);
stem(n, delta_M, 'filled', 'LineWidth', 2);
title(['ضربه با تاخیر M=' num2str(M)]);

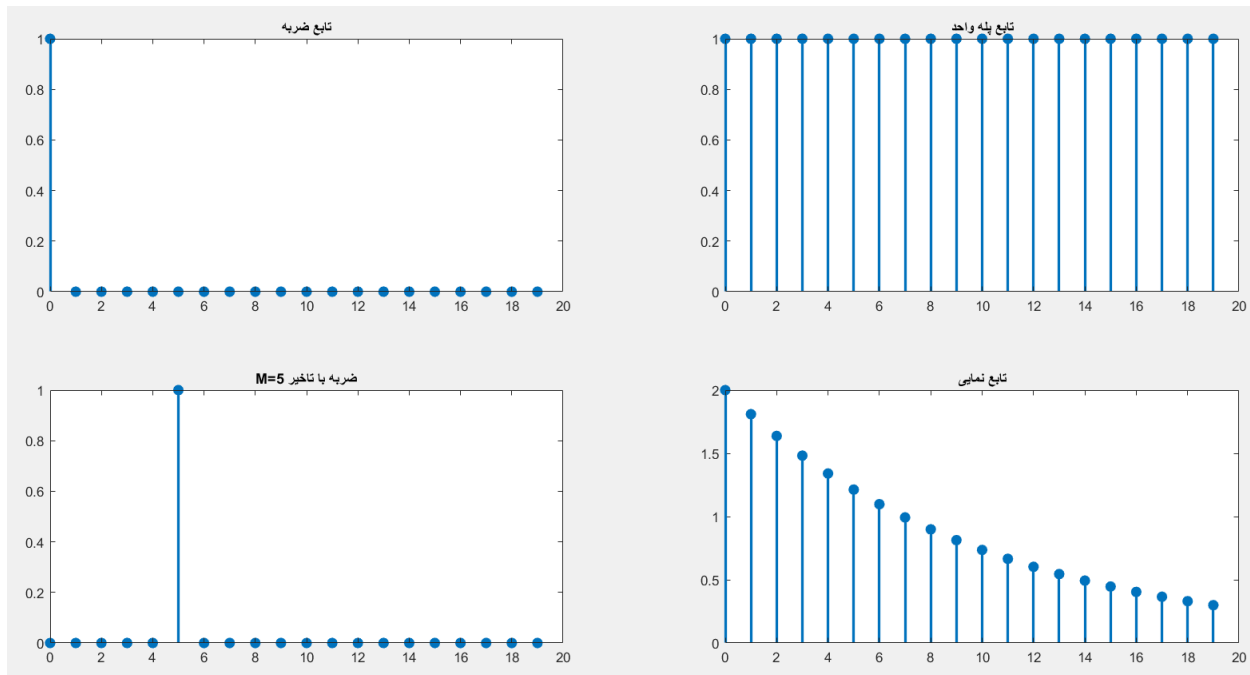
subplot(2,2,4);
stem(n, exponential, 'filled', 'LineWidth', 2);
title('تابع نمایی');

```

```

انرژی ضربه: 1
انرژی پله: 20
انرژی ضربه تاخیر یافته: 1
انرژی نمایی: 21.6625

```



فعالیت ۱۰: یک سیگنال کسینوسی یک بار با فرکانس 0.9 و یکبار با فرکانس 1.1 ایجاد کرده و با فاز ۹۰ درجه و در یک تصویر واحد همراه با legend ترسیم کرده و توان متوسط این سیگنال‌ها را محاسبه کنید.

$$X = A * \cos(2 * \pi * f * n + \varphi) \quad \text{و} \quad \text{دوره تناوب} = \frac{1}{f}$$

```
fs = 10;
t = 0:1/fs:10;
A = 1;

f1 = 0.9;
signal1 = A * cos(2 * pi * f1 * t);

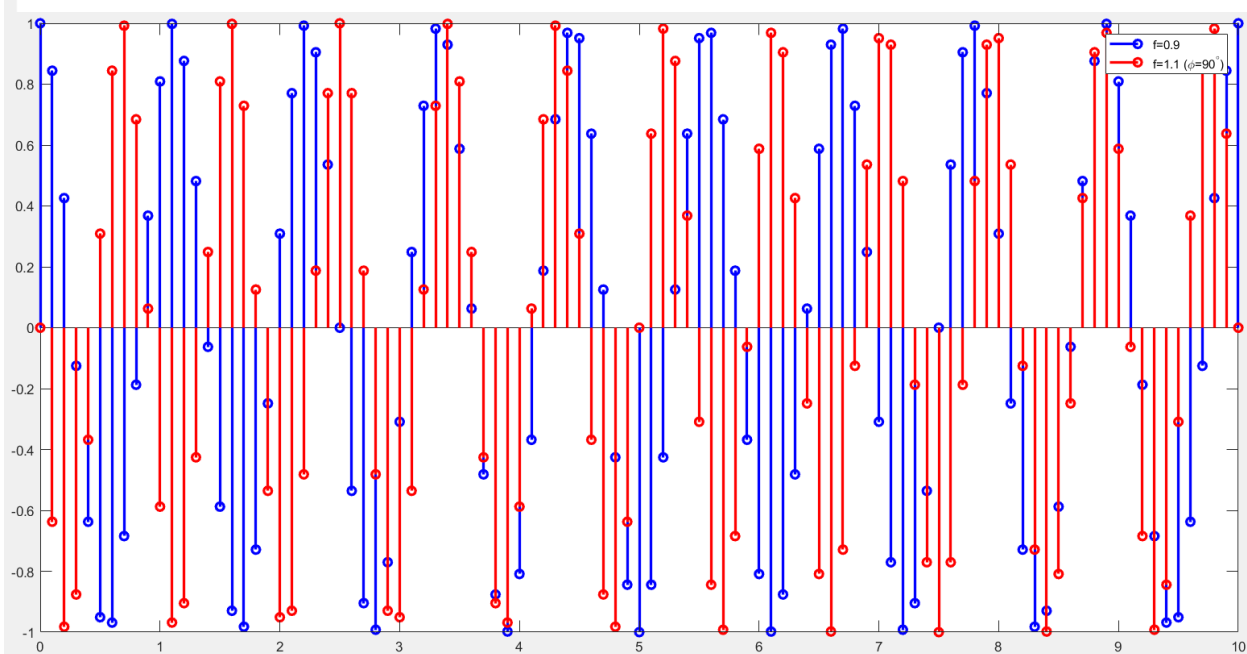
f2 = 1.1;
phase_shift = pi/2;
signal2 = A * cos(2 * pi * f2 * t + phase_shift);

figure;
stem(t, signal1, 'b', 'DisplayName', 'Signal 1', 'LineWidth', 2);
hold on;
stem(t, signal2, 'r', 'DisplayName', 'Signal 2', 'LineWidth', 2);
legend('f=0.9', 'f=1.1 (\phi=90^\circ)');

mean_power1 = mean(signal1.^2);
mean_power2 = mean(signal2.^2);

disp(['۱: توان متوسط سیگنال: ', num2str(mean_power1)]);
disp(['۲: توان متوسط سیگنال: ', num2str(mean_power2)]);
```

توان متوسط سیگنال ۱: 0.50495
توان متوسط سیگنال ۲: 0.49505

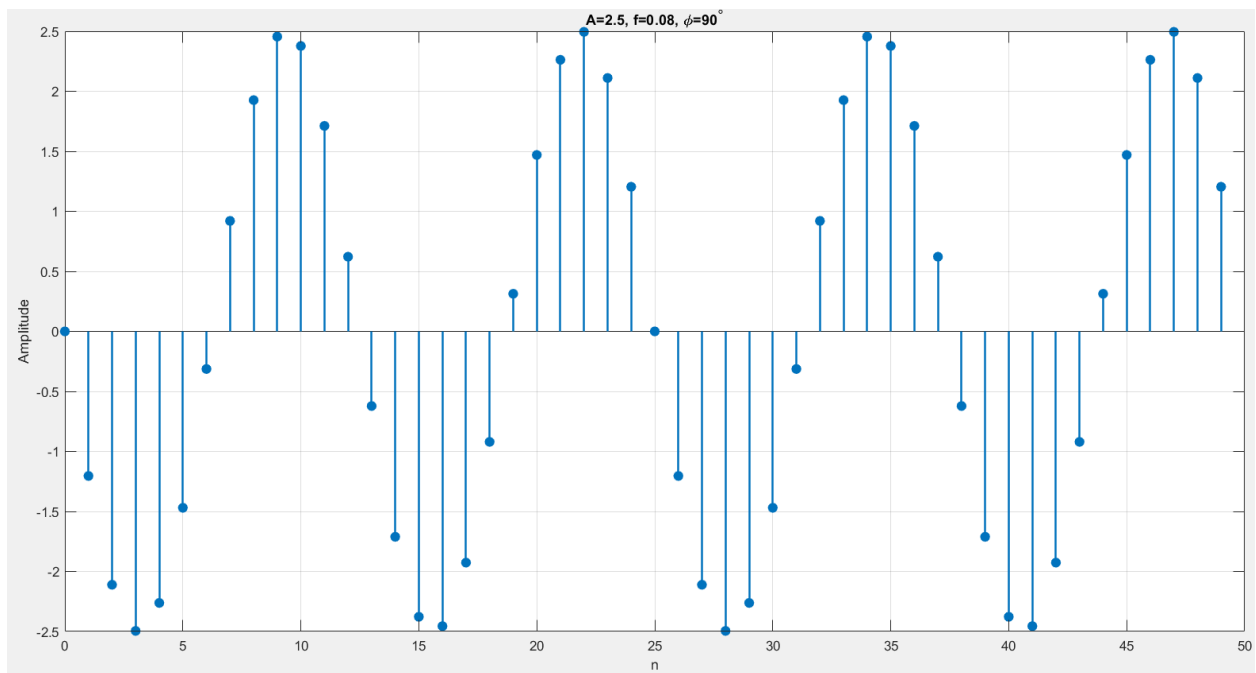


فعالیت ۱۱: یک برنامه بنویسید که سیگنال سینوسی با طول ۵۰، فرکانس ۰.۰۸، دامنه ۲.۵ و فاز ۹۰ درجه ایجاد کرده و آن را ترسیم کنید. دوره تناوب این سیگنال چند است؟

```
N = 50;
f = 0.08;
A = 2.5;
phi = pi/2;

n = 0:N-1;
x = A * cos(2 * pi * f * n + phi);

% رسم
figure;
stem(n, x, 'filled', 'LineWidth', 1.5);
title('A=2.5, f=0.08, \phi=90^\circ');
xlabel('n');
ylabel('Amplitude');
grid on;
```



تمرین ۵: کدام پارامتر، نرخ رشد و یا نزول این رشته را کنترل می‌کند؟ با کدام پارامتر می‌توان دامنه‌ی این سیگنال را تغییر داد؟

اگر سیگنال نمایی باشد، نرخ رشد یا نزول: توسط پارامتر C کنترل میشود. اگر بزرگتر از صفر باشد رشد نمایی است، اگر کوچکتر از صفر باشد رشد میرایی است و اگر برابر صفر باشد سیگنال ثابت خواهد بود.

همچنین در سیگنال نمایی، دامنه سیگنال توسط پارامتر A تعیین میشود. همچنین اگر $n=0$ باشد سیگنال برابر A خواهد بود.

تمرین ۸: نرخ رشد و نزول و دامنه‌ی این سیگنال توسط چه متغیرهایی تغییر می‌کند؟

در سیگنال سینوسی، دامنه توسط پارامتر A تعیین میشود و نرخ نوسان توسط پارامتر f تعیین میشود همچنین سیگنال رشد و نزول دائمی ندارد فقط نوسان میکند.

تمرین ۹: تفاوت بین عملگر $^$ با $^.$ در چیست؟

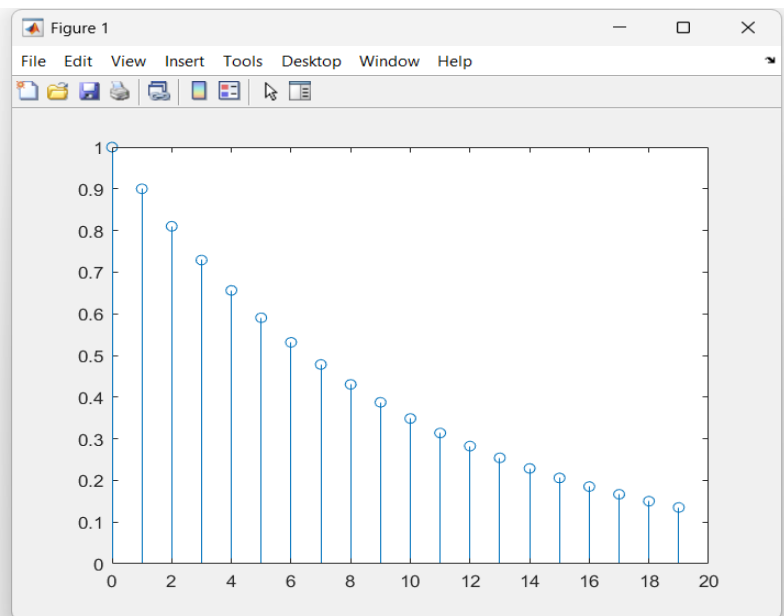
علامت $^$ عملگر ماترسی است و برای محاسبه عمل توان در ماتریس است. برای مثال در X^2 باید X از نوع یک ماتریس مربعی باشد تا عملیات ضرب $X * X$ انجام شود.

علامت $^.$ عملگر توان عنصر به عنصر است و این عملگر هر درایه از ماتریس یا بردار را جداگانه به توان میرساند.

تمرین ۱۰: پارامتر a را به عددی کمتر از یک تغییر دهید، چه تغییری در سیگنال حاصل می‌شود؟ برنامه را به ازای $a=0.9$ و $k=20$ اجرا کنید.

در سیگنال نمایی، اگر $a < 1$ باشد با افزایش n مقدار a^n به سمت صفر میل میکند و سیگنال نزولی (میرا) میشود.

```
>> k = 20; n = 0:k-1; a = 0.9;  
x = a.^n;  
stem(n, x);  
>>
```



تمرین ۱۱: طول سیگنال چند است و چگونه میتوان آن را تغییر داد؟

دوره تناوب سیگنال، از آنجایی که $T = \frac{1}{f} = 12.5$ بدست می آید و اعشاری است دوره تناوب سیگنال دو برابر آن یعنی هر ۲۵ سمپل است و از نمودار نیز این مشخص است.

طول سیگنال در کد برابر N است. برای تغییر طول سیگنال این عدد را کافی است تغییر دهیم.

تمرین ۱۲: انرژی سیگنال را یک بار در حالت اولیه‌ی تمرین P1-3 و دفعه‌ی دیگر بعد از اعمال تغییرات تمرین ۱۰ محاسبه کنید. چه تفاوتی در انرژی سیگنال ایجاد می‌شود؟ چرا؟

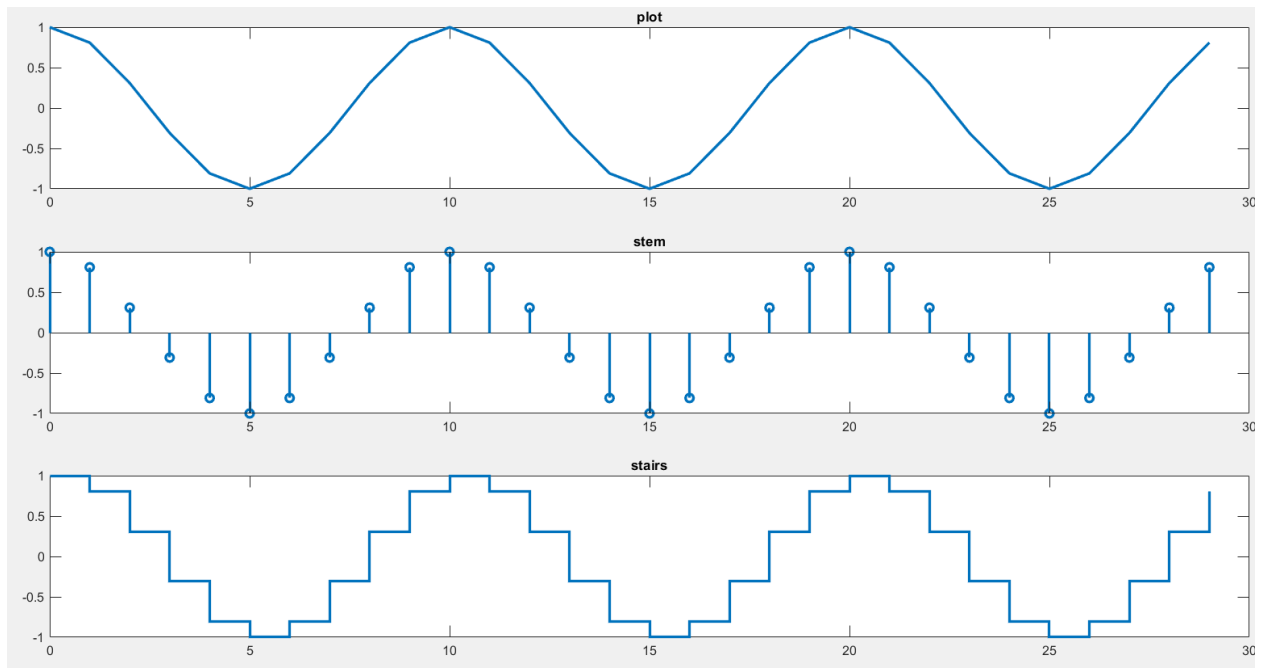
انرژی سیگنال با $a=0.9$ به طول قابل توجهی کمتر از سیگنال با $a=1$ و $a=1.1$ است.

-۱۲

فعالیت ۱۲: فرمان `stem` را یک نرتبه با فرمان `plot` و دفعه دیگر با فرمان `stairs` جایگزین کنید. شکل نمایش داده شده را در هر یک از این حالت‌ها با هم مقایسه کنید.

```
N = 30;
n = 0:N-1;
signal = cos(2*pi*0.1*n);

figure;
subplot(3,1, 1); plot(n, signal, 'LineWidth', 2); title('plot');
subplot(3,1, 2); stem(n, signal, 'LineWidth', 2); title('stem');
subplot(3,1, 3); stairs(n, signal, 'LineWidth', 2); title('stairs');
```



دستور plot نقاط را به هم متصل میکند و سیگنال پیوسته به نظر میرسد.

دستور stem به نقاط را به صورت میله ای نمایش میدهد.

دستور stairs نقاط را به صورت پله ای نمایش میدهد.

-۱۳

فعالیت ۱۳: برنامه ای بنویسید که هم یک سیگنال تصادفی با طول ۱۰۰ و با توزیع یکنواخت در بازه‌ی $[-2, 2]$ و هم یک سیگنال تصادفی به طول ۷۵ با توزیع نرمال و میانگین صفر و واریانس ۳ ایجاد کند و هر دو را در قالب یک پنجره نمایش دهد.

$$R = \mu + (\text{rand}(1,100)) \times \sigma$$

```
clc; clear; close all;
```

```
N1 = 100;
```

```
N2 = 75;
```

```
signal_uniform = -2 + 4 * rand(1, N1);
```

```
signal_normal = sqrt(3) * randn(1, N2);
```

```
N_max = max(N1, N2);
```

```
signal_uniform_padded = [signal_uniform, zeros(1, N_max - N1)];
```

```
signal_normal_padded = [signal_normal, zeros(1, N_max - N2)];
```

```
figure;
```

```
plot(signal_uniform_padded, 'b', 'DisplayName', 'Signal Uniform');
```

```
hold on;
```

```
plot(signal_normal_padded, 'r', 'DisplayName', 'Signal Normal');
```

```
legend show;
```

```
title('Random Signals');
```

```

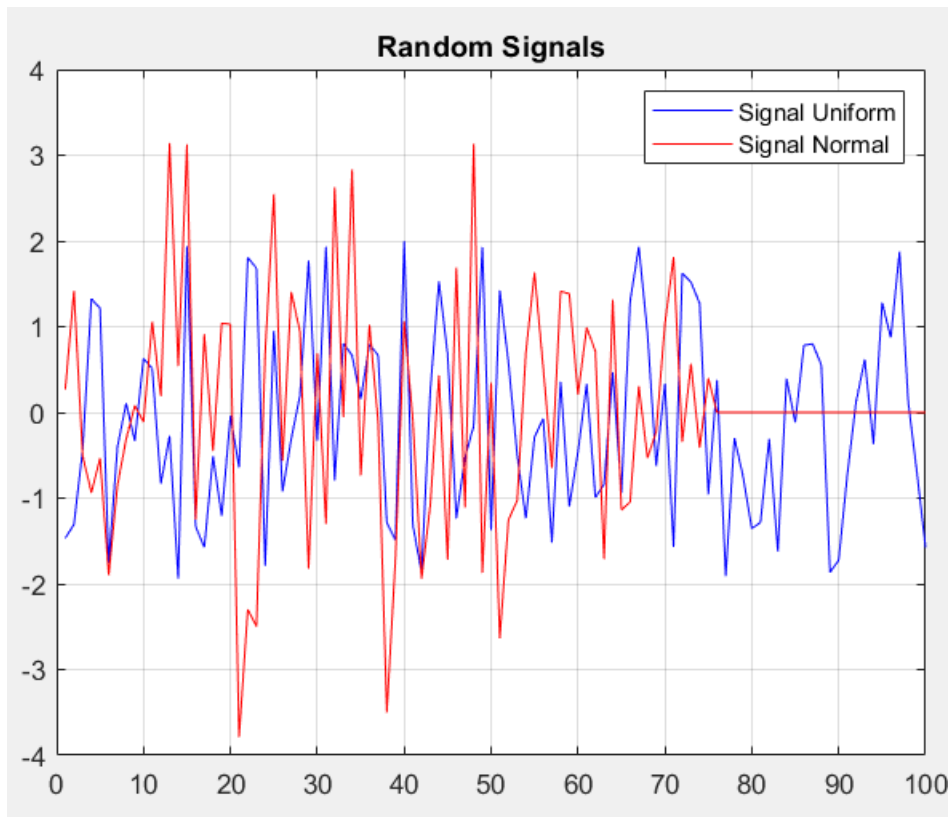
grid on;

fprintf('signal_uniform_padded:\n')
f14(signal_uniform_padded);
fprintf('\nsignal_normal_padded:\n');
f14(signal_normal_padded);

signal_uniform_padded:
miangin: -0.097665
variance: 1.271

signal_normal_padded:
miangin: 0.011911
variance: 1.6679
>>

```



-۱۴

فعالیت ۱۴: تابعی بنویسید که میانگین و واریانس دیتای داده شده به آن را محاسبه کند. سپس دو بردار تولید شده در فعالیت قبل را به آن بدهید و نتیجه را گزارش کنید.

```

function [mean_value, var_value] = f14(signal)
    mean_value = mean(signal);
    var_value = var(signal);

    disp(['miangin: ' num2str(mean_value)]);
    disp(['variance: ' num2str(var_value)]);
end

```

فعالیت ۱۵: برنامه‌ای بنویسید که یک سیگنال تصادفی سینوسی با طول ۵۰ را ایجاد کرده که فاز و دامنه‌ی آن به ترتیب بین بازه‌ی ۰ تا 2π و ۰ تا ۸۰ باشد که هر دو مستقل از هم و دارای توزیع یکنواخت باشند. سپس به دلخواه ۵ نمونه از این سیگنال را انتخاب کرده و نمایش دهید.

$$x[n] = A * \cos(w_0 * n + \varphi)$$

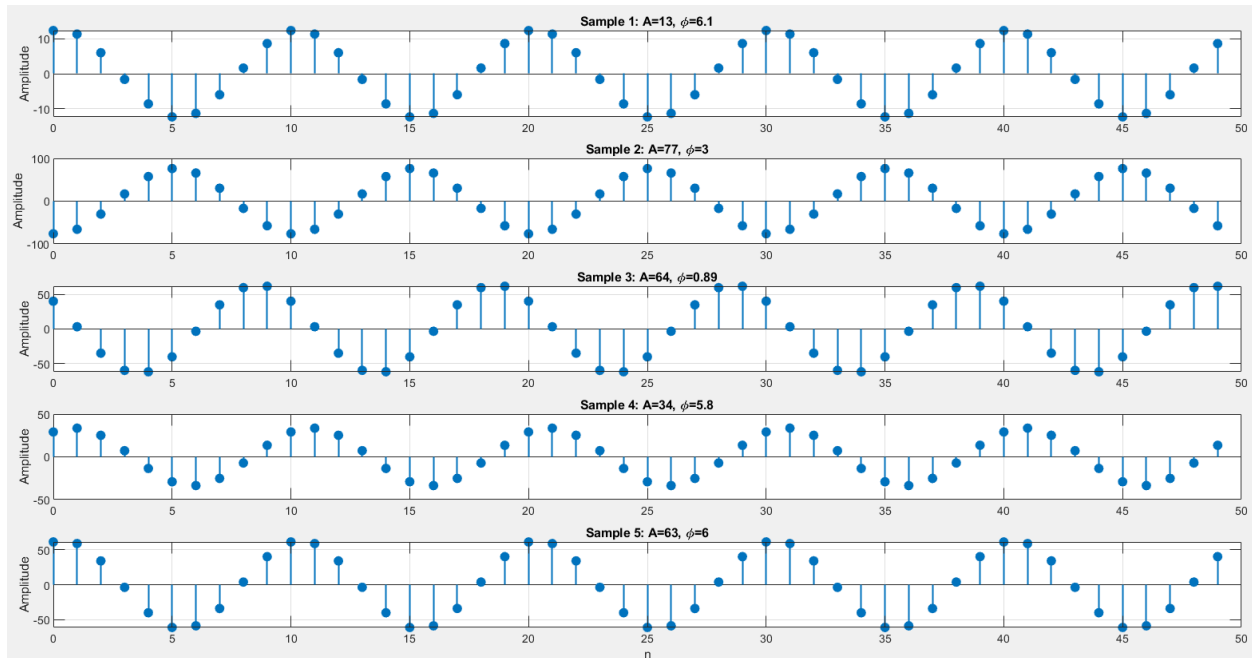
```
N = 50;
n = 0:N-1;
f = 0.1;

n_samples = 5;

figure;
for k = 1:n_samples
    A = rand(1) * 80;
    phi = rand(1) * 2*pi;

    x = A * cos(2 * pi * f * n + phi);

    subplot(5,1,k);
    stem(n, x, 'filled', 'LineWidth', 1.2);
    title(['Sample ', num2str(k), ': A=', num2str(A,2), ', \phi=', num2str(phi,2)]);
    ylabel('Amplitude');
    if k == 5, xlabel('n'); end
    grid on;
end
```



فعالیت ۱۶: برنامه‌ای بنویسید که سیگنال نمایی به طول ۵۱ و بصورت عبارت مقابل دریافت کرده:

$$Sig = 2n(0.9^n)$$

سپس با نویزی که دارای میانگین صفر و انحراف معیار ۰.۸ و توزیع یکنواخت است جمع کرده، در نهایت این سیگنال بعنوان ورودی به سیستم زیر وارد شود، خروجی را محاسبه کرده و نمایش دهید.

$$y[n] = \frac{1}{3} (x(n-1) + x(n) + x(n+1))$$

```
N = 51;
n = 0:N-1;
sig = 2 * n .* (0.9).^n;

noise = 0.8 * randn(size(n));
noisy_sig = sig + noise;

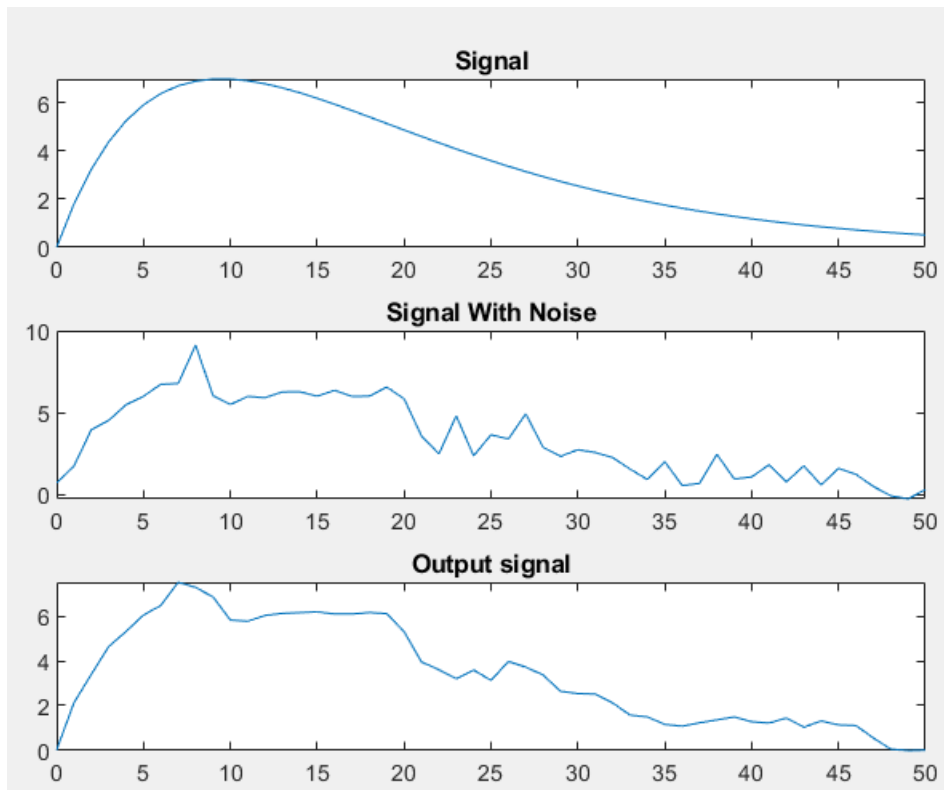
% y[n] = (1/3) * (x[n-1] + x[n] + x[n+1])
y = zeros(size(n));
for n_idx = 2:N-1
    y(n_idx) = (1/3) * (noisy_sig(n_idx-1) + noisy_sig(n_idx) + noisy_sig(n_idx+1));
end

figure;
grid on;

subplot(3,1,1);
plot(n, sig);
title('Signal');

subplot(3,1,2);
plot(n, noisy_sig);
title('Signal With Noise');

subplot(3,1,3);
plot(n, y);
title('Output signal');
```



آیا سیستم علی است؟ توضیح دهید.

خیر چون برای محاسبه خروجی به $x[n+1]$ یعنی یک نمونه از آینده نیازمندیم.

این سیستم چه کاری انجام میدهد؟

این سیستم یک نوع فیلتر است با طول پنجره ۳ و نویز را کاهش میدهد در نمودار نیز این موضوع مشخص است.

سیگنال سینوسی فعالیت قبل را نیز به عنوان ورودی به سیستم فوق وارد کرده و نتیجه را نمایش دهید.

```
N = 50;
n = 0:N-1;
f = 0.1;

n_samples = 5;

figure;
for k = 1:n_samples
    A = rand(1) * 80;
    phi = rand(1) * 2*pi;

    x = A * cos(2 * pi * f * n + phi);
```



```

y = zeros(size(n));
for n_idx = 2:N-1
    y(n_idx) = (1/3) * (x(n_idx-1) + x(n_idx) + x(n_idx+1));
end

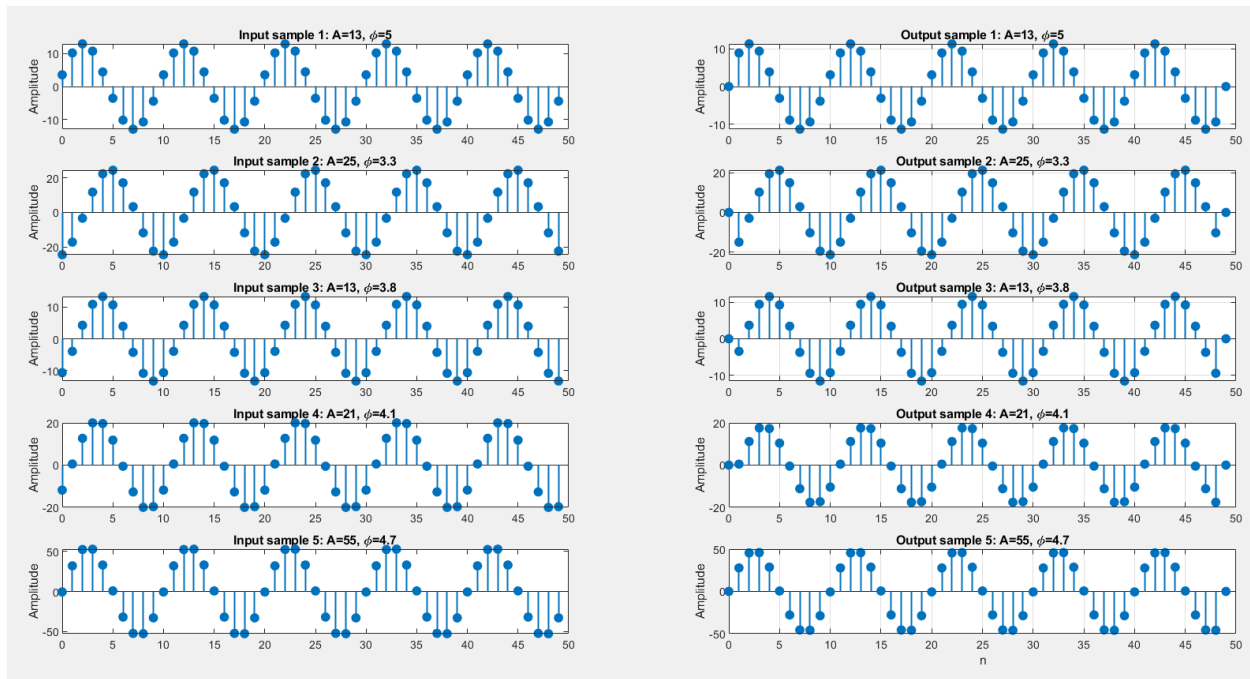
subplot(5,2, 2*k-1);
stem(n, x, 'filled', 'LineWidth', 1.2);
title(['Input sample ', num2str(k), ': A=', num2str(A,2), ', \phi=',
num2str(phi,2)]);
ylabel('Amplitude');

subplot(5,2, 2*k);
stem(n, y, 'filled', 'LineWidth', 1.2);
title(['Output sample ', num2str(k), ': A=', num2str(A,2), ', \phi=',
num2str(phi,2)]);
ylabel('Amplitude');

if k == 5, xlabel('n'); end
grid on;

end

```



آیا میتوانید با تغییر طول فیلتر (سیستم دیجیتالی) نتیجه بهتری بدست آورد؟

بله هرچه طول پنجره بیشتر شود نویز بیشتری نیز حذف میشود اما لبه ها و تغییرات سریع سیگنال نیز از بین میرود.

فعالیت ۱۷: فرکانس لحظه‌ای در هر سیگنال سینوسی برابر است با مشتق فاز آن نسبت به زمان: برای ایجاد سیگنال‌هایی که فرکانس آن‌ها با زمان به صورت خطی تغییر میکند، نیاز است تا آرگومان آن‌ها تابعی از درجه‌ی دو نسبت به زمان باشد.

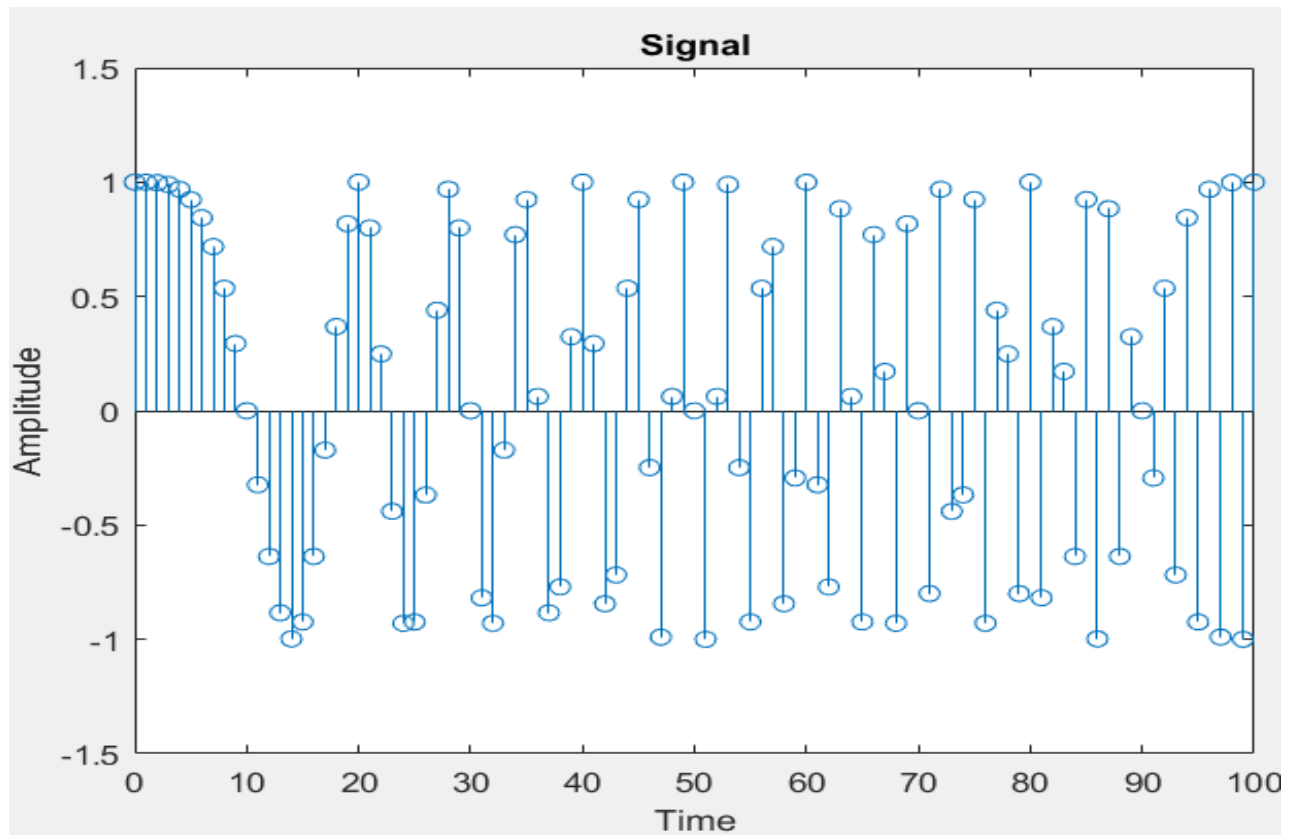
$$P = a \times n^2 + b \times n \Rightarrow f = \frac{dP}{dn} = 2an + b$$

مثالی از متغیر بودن فرکانس:

```
n = 0:100;
a = pi/200; b = 0;
arg = a * n.* n + b * n;
x = cos(arg);
stem(n,x)
axis([0,100,-1.5,1.5]);
title('Signal')
xlabel('Time')
ylabel('Amplitude')
grid; axis;
```

```
n = 0:100;
a = pi/200; b = 0;

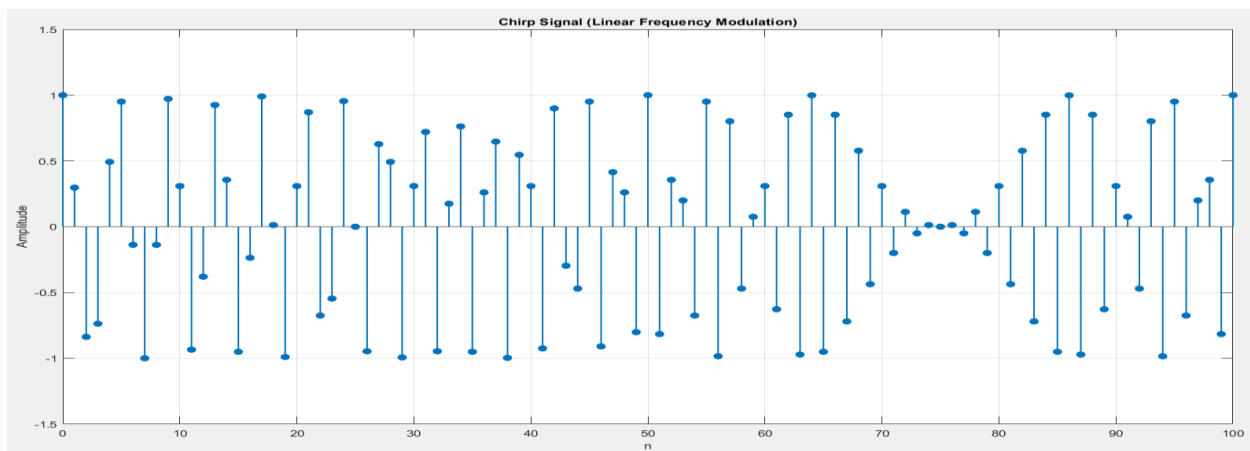
arg = a* n .* n + b * n;
x = cos(arg);
stem(n, x);
axis([0, 100, -1.5, 1.5]);
title('Signal');
xlabel('Time');
ylabel('Amplitude');
```



```
n = 0:100;
a = 0.002;
b = 0.2;

arg = 2 * pi * (a * n.^2 + b * n);
x = cos(arg);

stem(n, x, 'filled', 'LineWidth', 1.2);
axis([0, 100, -1.5, 1.5]);
title('Chirp Signal (Linear Frequency Modulation)');
xlabel('n');
ylabel('Amplitude');
grid on;
```



فعالیت ۱۸: برنامه را طوری تغییر دهید که سیگنال سینوسی با فرکانس متغیر با مینیمم ۰.۲ و ماکزیمم ۰.۶ ایجاد کند.

```
N = 100;
f_min = 0.2;
f_max = 0.6;

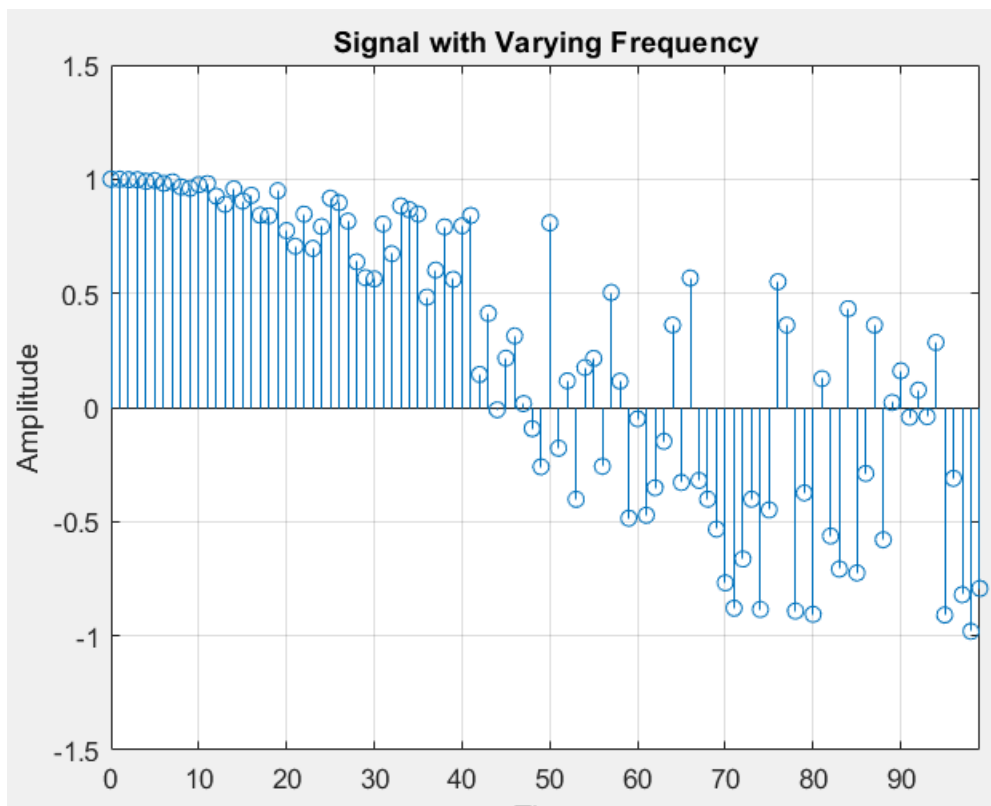
frequencies = (f_max - f_min) * rand(N, 1) + f_min;

n = 0:N-1;
t = n / N;

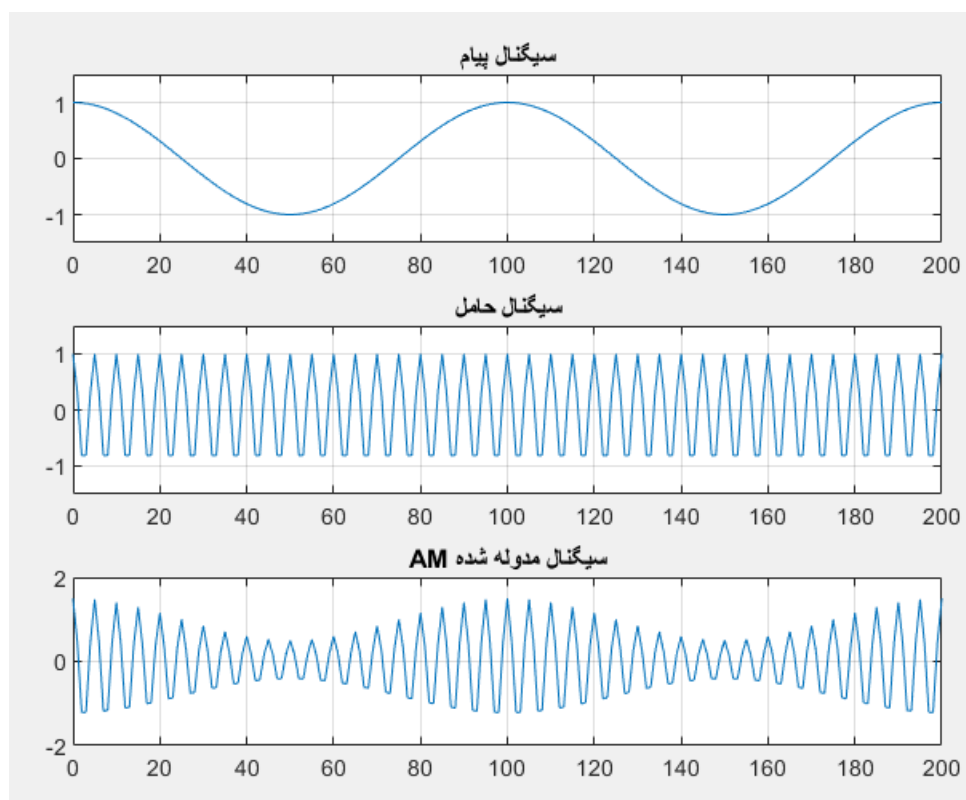
x = zeros(1, N);
for i = 1:N
    arg = 2 * pi * frequencies(i) * t(i);
    x(i) = cos(arg);
end

stem(n, x);
axis([0, N-1, -1.5, 1.5]);
title('Signal with Varying Frequency');
xlabel('Time');
ylabel('Amplitude');

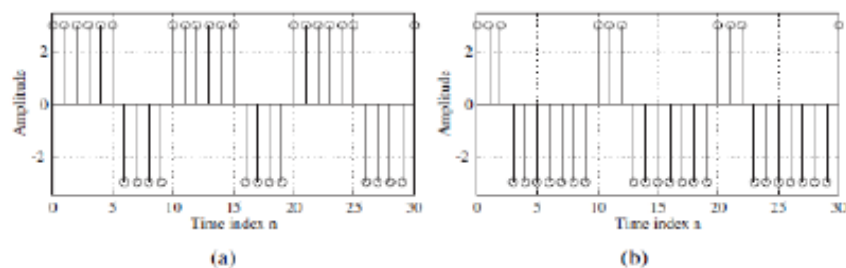
grid on;
```



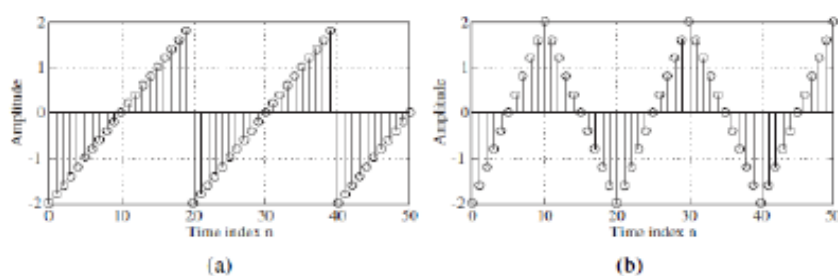
فعالیت ۱۹: برنامه ای بنویسید که به ازای مقادیر مختلف اندیس مدولاسیون، فرکانس حامل و فرکانس سیگنال اصلی شکل موج را ترسیم کند، سپس نتیجه را تحلیل کنید.



فعالیت ۲۰: با استفاده از توابع square و sawtooth، برنامه‌ای بنویسید که خروجی آن، نمودارهای زیر باشد.



شکل ۱. شکل موج مربعی



```

t = linspace(0, 6*pi - 1, 100);

A = sawtooth(t) - sawtooth(t-20) + 0.7;
B = sawtooth(t-20) - sawtooth(t) - 0.7;
C = sawtooth(t);
D = square(t) .* sawtooth(t) + 0.5;

figure; grid on;
subplot(4,1,1); stem(t, A, 'LineWidth', 2);
subplot(4,1,2); stem(t, B, 'LineWidth', 2);
subplot(4,1,3); stem(t, C, 'LineWidth', 2);
subplot(4,1,4); stem(t, D, 'LineWidth', 2);

```

